SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	T. Deekshith	**Reg. No.:**	192425145
Date:		**Duration:**	60 minutes

Code of Conduct

I _T. Deekshith(192425145 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __t.Deekshith_____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.

- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

AI-Based Predictive Modelling for Early Forecasting of Bovine Mastitis in Indian Dairy Farms

1. Analyze System Requirements

1.1 Study of the Problem Statement

Bovine mastitis is an inflammatory disease affecting the mammary glands of dairy cattle. It is one of the major health problems in dairy farming because it can reduce milk production, affect milk quality, increase veterinary expenses, and cause economic losses to farmers. In many Indian dairy farms, mastitis detection depends mainly on manual observation and physical examination. These methods may not always identify early changes in an animal's health condition.

The proposed system uses Artificial Intelligence and Machine Learning to analyze animal health information and estimate the risk of mastitis. It collects information such as animal breed, age, milk production, body temperature, udder condition, lactation stage, and previous medical history. The collected information is processed and passed to a trained machine learning model. Based on the analysis, the system provides a risk category and alerts the farmer when further veterinary examination may be required.

1.2 System Objectives

The main objective of the proposed system is to develop an intelligent prediction platform that supports early identification of possible mastitis cases in dairy cattle. It aims to reduce dependence on manual observation by analyzing animal health information through machine learning techniques. The system also aims to maintain digital health records, provide risk notifications, support veterinary decision-making, and improve the overall management of dairy farms. It is designed to be simple, accessible, and suitable for different types of Indian dairy farms.

1.3 Functional Requirements

Functional requirements describe the operations that the proposed system must perform. The system should provide user registration and login facilities for farmers, veterinarians, and administrators. It should allow farmers to register individual animals and enter their health information, including milk yield, body temperature, breed, and udder condition. It should validate the entered information and store

it in the database. The system should process the collected data using a machine learning model and generate a mastitis risk prediction. It should display low, medium, and high risk categories, provide alerts for high-risk predictions, and maintain previous prediction records. It should also generate reports that help farmers and veterinarians review animal health conditions.

1.4 Non-Functional Requirements

Non-functional requirements describe the quality and performance expectations of the proposed system. The application should provide a simple and user-friendly interface so that farmers can operate it without advanced technical knowledge. It should process animal information and generate prediction results within a reasonable time. The system should protect user accounts and animal health records through authentication and access control. It should be reliable when storing and retrieving information and should support future expansion when the number of registered animals increases. The application should also be maintainable so that developers can update the prediction model, database, and user interface without redesigning the entire system.

1.5 Key Quality Attributes

The proposed system should maintain scalability, maintainability, reliability, availability, performance, and security. Scalability allows the application to support more users and animal records as the number of participating dairy farms increases. Maintainability makes it easier to modify individual modules and update the machine learning model. Reliability ensures that animal records and prediction results are processed correctly. Availability helps farmers access the application whenever they need to review animal information. Performance ensures that the system provides results without unnecessary delays. Security protects confidential user information and prevents unauthorized access to stored records.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Layered Architecture with Client-Server Design

The proposed system uses a Layered Architecture with a Client-Server design. This architecture separates the application into different layers, where each layer performs a specific responsibility. The client-server approach allows farmers and veterinarians to access the application through a web browser

or mobile interface, while the server processes requests, communicates with the machine learning model, and manages stored information.

The architecture consists of a presentation layer, application or business logic layer, machine learning layer, and data layer. The presentation layer provides the interface for entering animal details and viewing prediction results. The application layer validates user information and coordinates the system operations. The machine learning layer processes the collected data and estimates the risk of mastitis. The data layer stores animal information, health observations, user details, and prediction history.

2.2 Presentation Layer

The presentation layer is the front-end part of the application through which users interact with the system. It can be developed using HTML, CSS, JavaScript, and React. Farmers can use this layer to register animals, enter daily health observations, and view mastitis prediction results. Veterinarians can access animal records and review prediction reports. The interface should use clear labels, simple forms, and understandable risk indicators to make the application easy to use.

2.3 Application Layer

The application layer acts as the central processing component of the system. It receives requests from the presentation layer, validates the submitted information, and manages communication between different modules. It handles user authentication, animal registration, health data management, prediction requests, and report generation. This layer can be implemented using Python Flask or Django. It also communicates with the database and machine learning layer to complete the required operations.

2.4 Machine Learning Layer

The machine learning layer is responsible for predicting the possibility of mastitis using animal health information. The collected data is first processed to handle missing values, inconsistent formats, and incorrect entries. The prepared data is then provided to a trained classification model. Algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine can be evaluated. The selected model generates a prediction that can be displayed as a low, medium, or high risk category. The model should be trained and tested using reliable animal health records before being used in the application.

2.5 Data Layer

The data layer is responsible for storing and managing all information required by the application. It contains user details, animal identification records, health observations, milk production information, prediction results, and veterinary reports. A relational database such as MySQL can be used to organize this information. Each animal should have a unique identification number to ensure that its records are stored and retrieved correctly. The database also maintains historical records for future analysis and comparison.

2.6 Justification for Selecting This Architecture

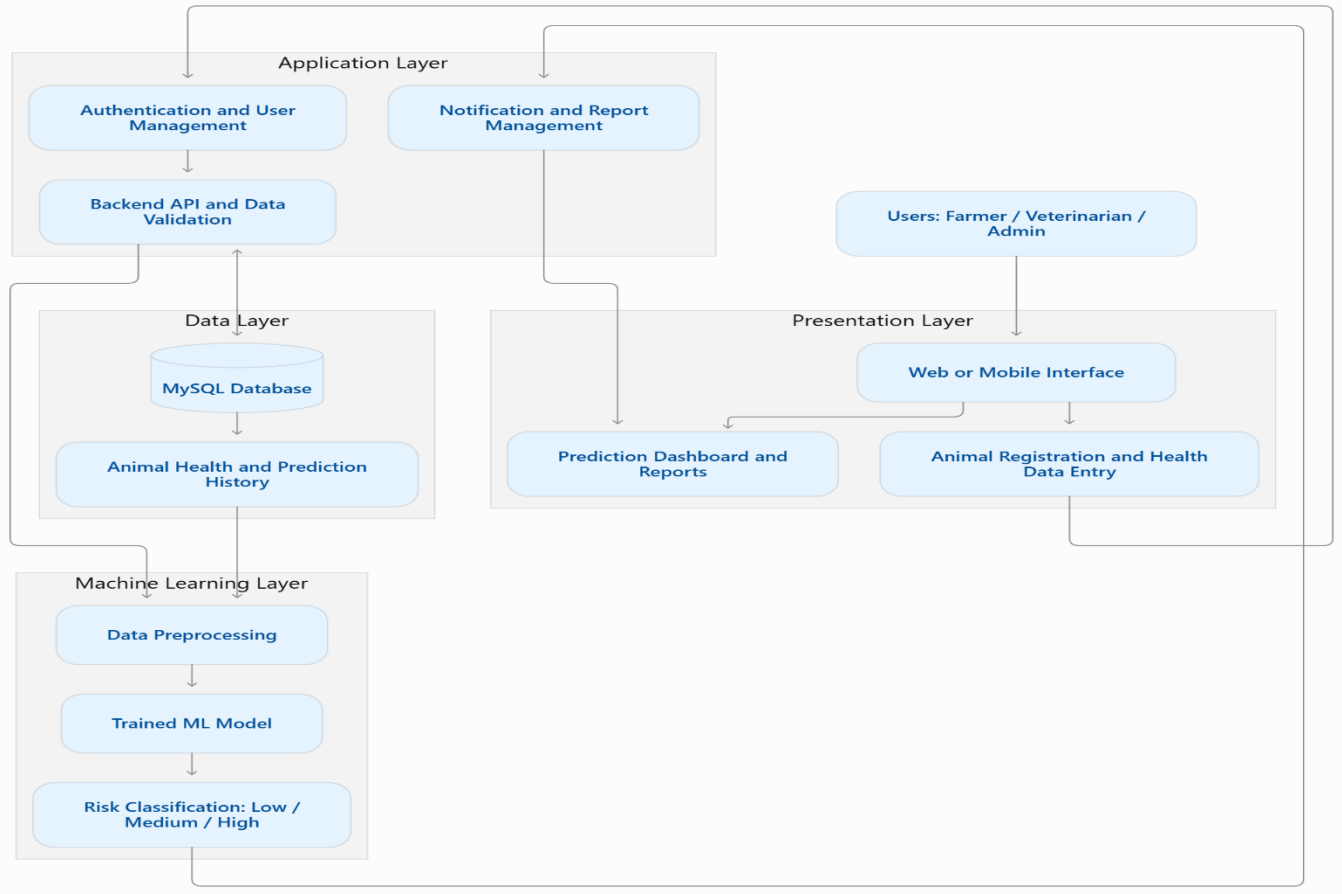
The layered architecture is suitable because the proposed system contains different responsibilities, including user interaction, data processing, machine learning prediction, and database management. Separating these responsibilities makes the application easier to understand, develop, test, and maintain. The client-server design allows multiple farmers and veterinarians to access the application through different devices. It also supports future improvements, such as adding IoT sensors, mobile applications, regional language support, and advanced prediction algorithms.

3. Draw the Software Architecture Diagram

3.1 Overall Architecture Diagram

The architecture diagram represents the overall structure of the proposed AI-based mastitis prediction system. It shows how farmers and veterinarians interact with the application, how animal information moves through the different layers, and how the machine learning model communicates with the database to generate prediction results.

Figure 1: Software Architecture of AI-Based Bovine Mastitis Prediction System



3.2 Explanation of the Architecture Diagram

The system begins when a farmer or veterinarian accesses the application through a web or mobile interface. The user enters animal information through the presentation layer, and the submitted information is transferred to the backend through an API. The application layer validates the information and communicates with the database to store the animal records. After validation, the machine learning layer preprocesses the information and passes it to the trained prediction model.

The prediction model analyzes the available health information and produces a risk classification. The application layer receives this result and displays it through the dashboard. If the prediction indicates high risk, the notification module can generate an alert recommending veterinary examination. The database maintains the animal details, health observations, and prediction history. This complete architecture provides an organized method for collecting, processing, storing, and analyzing dairy animal health information.

3.3 Technologies Used in the Architecture

The proposed system can use different technologies for implementing each architectural layer. The selection of technologies depends on the available development environment, project requirements, and dataset.

Architecture component	Suggested technology	Purpose
Frontend	HTML, CSS, JavaScript, React	User interface
Backend	Python Flask or Django	Application processing
Machine Learning	Scikit-learn	Mastitis risk prediction
Data Processing	Pandas and NumPy	Data cleaning and preparation
Database	MySQL	Storing animal and user records
API Communication	REST API	Communication between layers
Development Tool	VS Code	Writing and managing code

These technologies are proposed implementation choices. The final selection can be adjusted according to project requirements and available resources.

4. Explain Components and Their Interactions

4.1 User Interface Component

The user interface is the main interaction point between the application and its users. It allows farmers to enter animal details and access prediction results. The interface provides options for animal registration, health data entry, prediction history, and report viewing. It should be designed with simple navigation and clear information so that users can understand the system without specialized technical knowledge.

4.2 User Authentication Component

The authentication component manages user login and access permissions. Farmers, veterinarians, and administrators may have different access levels depending on their responsibilities. The system verifies login credentials before allowing access to protected information. This component helps prevent unauthorized users from viewing or modifying animal records.

4.3 Animal Data Management Component

This component collects and manages the information related to dairy animals. It stores animal identification, breed, age, lactation stage, milk yield, body temperature, udder condition, and previous medical history. The component also allows users to update animal information when new observations are available. The collected information becomes the input for the machine learning prediction process.

4.4 Data Preprocessing Component

The data preprocessing component prepares the collected information before it is analyzed by the machine learning model. It checks for missing values, duplicate records, inconsistent data formats, and incorrect entries. Categorical information may be converted into numerical values, and relevant features are selected for prediction. Proper preprocessing improves the quality of the input data and helps the model produce more consistent results.

4.5 Machine Learning Prediction Component

The machine learning component analyzes the processed animal information and estimates the possibility of mastitis. It uses a trained classification algorithm developed from suitable animal health records. The model can consider factors such as temperature, milk production, udder condition, and previous disease history. The output is converted into a risk category that helps users understand which animals may require additional attention.

4.6 Database Management Component

The database management component stores all important information required by the system. It maintains user accounts, animal details, health observations, prediction results, and historical records. The application communicates with the database whenever information needs to be inserted, updated, or retrieved. Organized database storage makes it easier to maintain records and generate reports.

4.7 Notification and Report Component

The notification component communicates important prediction results to the users. When an animal receives a high-risk classification, the system can display a warning on the dashboard or send a notification through an available communication service. The report component generates summaries of animal health records and prediction history. These reports help farmers and veterinarians review the condition of animals and plan further examination.

4.8 Overall Component Interaction and Workflow

The system workflow begins with user login and animal registration. After registration, the farmer enters the animal's health and production information. The application validates the information and stores it in the database. The preprocessing component prepares the data and sends it to the machine learning model. The model analyzes the information and returns a prediction result. The application displays the risk category on the dashboard and generates an alert when necessary. Finally, the prediction result is stored in the database for future reference. This interaction ensures that all components work together to provide a complete prediction service.

Table 1: Component Interaction

From component	To component	Information exchanged
User Interface	Backend API	User input and requests
Backend API	Database	Animal and user records
Backend API	Preprocessing	Validated animal data
Preprocessing	ML Model	Processed features
ML Model	Risk Classification	Prediction output
Risk Classification	Notification Module	Risk category
Database	Report Module	Stored health records
Report Module	User Interface	Reports and summaries

5. Discuss Quality Attributes

Quality attributes describe how effectively the proposed software architecture performs under different conditions. The system should be designed to provide secure, reliable, fast, and continuous

services to farmers and veterinarians. The following quality attributes are important for the successful implementation of the proposed system.

5.1 Scalability

Scalability refers to the ability of the application to handle an increasing number of users, animals, and health records. Initially, the system may be used by a small number of dairy farms. In the future, it may be expanded to support larger farms, veterinary hospitals, and multiple dairy regions. The layered client-server architecture supports this expansion because additional server resources and database capacity can be introduced when required. The system can also be extended to process information from IoT sensors and other digital farm equipment.

5.2 Security

Security ensures that animal health information and user details are protected from unauthorized access. The system should provide secure login facilities and role-based access for farmers, veterinarians, and administrators. Passwords should be stored securely, and communication between the frontend and backend should use HTTPS. Database access should be restricted to authorized application components. Regular backups and appropriate access controls can help protect important information from accidental loss or misuse.

5.3 Performance

Performance refers to the speed at which the application processes requests and displays results. The system should provide a quick response when farmers enter animal details or request mastitis predictions. Data preprocessing should be optimized so that unnecessary operations do not delay the prediction process. Database queries should also be designed efficiently to retrieve animal records and prediction history. A properly trained machine learning model can provide predictions within a reasonable time, supporting regular farm monitoring.

5.4 Reliability

Reliability refers to the ability of the system to perform its functions correctly and consistently. The application should store animal information accurately and produce predictions using validated input data. Error handling should be implemented to manage invalid entries, database connection

problems, and unexpected application failures. Regular database backups can reduce the risk of losing important records. The machine learning model should also be evaluated using suitable test data to understand its prediction reliability.

5.5 Maintainability

Maintainability refers to how easily the application can be modified, corrected, and improved. The selected layered architecture separates the frontend, backend, machine learning model, and database responsibilities. This separation allows developers to update one component without making unnecessary changes to the others. For example, a new machine learning algorithm can be introduced without completely redesigning the user interface. Developers can also improve existing modules, correct errors, and add new features according to user requirements.

5.6 Availability

Availability refers to the ability of users to access the application whenever required. Farmers may need to check animal health records and prediction results during daily farm operations. The application can be deployed on a suitable hosting platform so that authorized users can access it through internet-connected devices. Database backups, server monitoring, and recovery procedures can help reduce service interruptions. Future versions may also include limited offline data entry with synchronization when internet connectivity becomes available.

Table 2: Quality Attribute Analysis

Quality attribute	How the architecture supports it
Scalability	Supports additional users, animals, and server resources
Security	Uses authentication, access control, and secure communication
Performance	Uses optimized processing and database queries
Reliability	Provides validation, error handling, and data backups
Maintainability	Separates the application into independent layers
Availability	Supports hosted access and recovery procedures

6. Justify Architectural Decisions

6.1 Rationale Behind Architectural Choices

The layered client-server architecture is selected because it provides a clear separation between the different responsibilities of the proposed application. The presentation layer manages user interaction, the application layer handles business operations, the machine learning layer performs prediction, and the data layer stores information. This structure makes the overall system easier to develop, understand, and maintain. It also supports communication between farmers, veterinarians, and the prediction system through a centralized application.

The proposed architecture is suitable for a project that combines web application development with machine learning. The machine learning model can be developed and tested separately before being connected to the backend through an API. Similarly, the database can be managed independently from the frontend. This separation reduces complexity and allows individual components to be improved whenever necessary.

6.2 Trade-Offs Considered While Selecting the Architecture

Every software architecture has certain advantages and limitations. The layered architecture provides a simple and organized development structure, but communication between layers may introduce some processing overhead. A centralized backend makes information management easier, but the application may depend heavily on server availability. A client-server design also requires appropriate internet connectivity for online access.

Microservices architecture was another possible choice because it separates an application into independently deployable services. However, it introduces additional complexity in deployment, communication, monitoring, and maintenance. For the proposed academic project, a layered architecture with client-server design is more manageable and suitable. It provides sufficient separation of responsibilities without requiring the additional infrastructure of a large distributed system.

Table 3: Architectural Alternatives and Trade-Offs

Architecture	Advantages	Limitations	Suitability
Layered Architecture	Simple, organized, and maintainable	Communication between layers may add overhead	Highly suitable
Microservices	Independent deployment and scaling	More complex to develop and manage	Suitable for large systems
MVC Architecture	Separates model, view, and controller	Does not alone define the complete deployment architecture	Suitable for the user interface
Event-Driven Architecture	Supports asynchronous notifications	Requires additional event management	Useful for future alerts
Serverless Architecture	Reduces server management	Depends on platform services and usage limits	Possible future option

6.3 Support for Future Enhancements

The proposed architecture can support several future improvements without requiring a complete redesign. IoT sensors can be integrated to collect animal health information automatically, including body temperature and activity-related measurements. A mobile application can be introduced to allow farmers to access the system through smartphones. Regional language support can make the application easier to use in different parts of India.

The machine learning component can also be improved by training new models using larger and more diverse datasets collected from different dairy farms. Additional algorithms and advanced prediction techniques can be evaluated to improve performance. The system may also include graphical reports, farm-level analytics, and automated health monitoring features. These enhancements can make the application more useful for both small and large dairy farms.

6.4 System Integration

The system can integrate with external services through APIs. For example, a notification service can be connected to send alerts to farmers when a high-risk prediction is generated. IoT devices can be connected to collect animal health measurements automatically. Veterinary record systems may also be integrated in the future to support information sharing, subject to permission and compatibility. These integrations can improve the usefulness of the application and reduce manual

data entry.

6.5 Long-Term Maintainability

Long-term maintainability is supported by keeping the software components separate and using standard development technologies. The frontend, backend, machine learning model, and database can be updated independently wherever possible. Developers can monitor system performance, correct errors, update security measures, and improve the prediction model as new verified data becomes available. Proper documentation and testing should be maintained throughout development. This approach helps the system remain useful and adaptable as dairy farming requirements change.

Overall Conclusion

The proposed AI-Based Predictive Modelling System for Early Forecasting of Bovine Mastitis in Indian Dairy Farms uses a layered client-server architecture to organize user interaction, application processing, machine learning prediction, and database management. The architecture supports the major functional and non-functional requirements of the system and provides a structured method for processing animal health information.

The selected architecture offers advantages in scalability, security, performance, reliability, availability, and maintainability. Its modular structure also allows future integration with IoT devices, mobile applications, notification services, and advanced machine learning models. By providing early risk estimates and organized animal health records, the proposed system can support farmers and veterinarians in identifying animals that may require further examination. It is intended as a decision-support system and does not replace professional veterinary diagnosis.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	organized, professional report.		report needs improvement.	

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25